

BÀI 1. TỔNG QUAN VỀ .NET & KIẾN TRÚC PHẦN MỀM

Mục tiêu bài học: Sau khi hoàn thành tuần học này, bạn sẽ có khả năng:

1. **Trình bày tự tin** về hệ sinh thái .NET và lý do tại sao chúng ta chọn nó.
2. **Vẽ và giải thích** **rành mạch** luồng đi của một yêu cầu web trong mô hình MVC.
3. **Bảo vệ được quan điểm** tại sao kiến trúc 3 lớp lại quan trọng cho việc bảo trì và mở rộng dự án.
4. **Tự tay thiết lập** một Solution .NET hoàn chỉnh, có cấu trúc rõ ràng, bằng cả công cụ trực quan (Visual Studio) và dòng lệnh (Terminal).

NỘI DUNG LÝ THUYẾT CHI TIẾT

1.1. Giới thiệu .NET & ASP.NET Core

1.1.1. Lịch sử phát triển

Để hiểu rõ tại sao chúng ta lại lựa chọn công nghệ hiện tại, hãy cùng nhìn lại hành trình tiến hóa của .NET qua ba giai đoạn lịch sử quan trọng dưới đây. Từ một nền tảng mang tính khép kín, .NET đã lột xác thành một hệ sinh thái mở và mạnh mẽ bậc nhất.

1. Thời kỳ .NET Framework (2002 - ~2019): "Khu vườn có tường bao"

- **Đặc điểm cốt lõi:** Đây là nền tảng ban đầu do Microsoft phát triển. Dù sở hữu khả năng xử lý rất mạnh mẽ, nó lại mang nhược điểm lớn về tính linh hoạt và môi trường triển khai.
- Hệ thống này được ví như một "khu vườn có tường bao" biệt lập. Nó chỉ có thể phát triển và vận hành tốt nhất bên trong hệ sinh thái Windows của Microsoft.
- Nếu doanh nghiệp xây dựng một ứng dụng web bằng .NET Framework, ứng dụng đó gần như bắt buộc phải chạy trên hệ điều hành Windows. Bức tường công nghệ này ngăn cản việc mang ứng dụng sang vận hành ở các hệ điều hành khác.

2. Thời kỳ .NET Core (2016 - 2020): Cuộc cách mạng "Đập đi xây lại"

- **Đặc điểm cốt lõi:** Đánh dấu một bước ngoặt mang tính lịch sử khi Microsoft quyết định thay đổi hoàn toàn tư duy và triết lý phát triển phần mềm.
- Thay vì vá vú trên nền tảng cũ, Microsoft đã thực hiện một cuộc cách mạng: "đập đi xây lại" để tạo ra một phiên bản hoàn toàn mới. Triết lý chủ đạo của thời kỳ này gói gọn trong hai từ: **tự do và cởi mở**.
 - **Lúc lập trình:** Bạn có thể tự do code trên máy MacBook (macOS), đồng nghiệp của bạn có thể code trên máy tính Windows.
 - **Lúc triển khai:** Sản phẩm cuối cùng có thể đưa thẳng lên một server chạy **Linux** – vốn được coi là "ngôi nhà" trú ngụ của hầu hết các hệ thống máy chủ lớn trên thế giới hiện nay. Mọi thứ vẫn hoạt động trơn tru mà không gặp rào cản nào.

3. Thời kỳ .NET 5, 6, 7, 8... (2020 - nay): Kỷ nguyên Hợp nhất và Hiện đại hóa

- **Đặc điểm cốt lõi:** Microsoft chính thức xóa bỏ sự phân tách phức tạp trước đây, hợp nhất hai thế giới (.NET Framework và .NET Core) làm một và gọi tên đơn giản là **.NET**.
- Chúng ta đang được sử dụng phiên bản .NET hiện đại, hợp nhất toàn diện và mang lại hiệu năng mạnh mẽ nhất. Trong kỷ nguyên này, bạn cần đặc biệt lưu ý quy luật phân loại phiên bản:
 - Các phiên bản mang số chẵn (ví dụ: .NET 6, .NET 8) được gọi là LTS (Long-Term Support).
 - **Ví dụ cụ thể về ứng dụng doanh nghiệp:** Nhờ tính chất **cực kỳ ổn định** và **được hỗ trợ kỹ thuật trong thời gian dài**, các phiên bản LTS như .NET 6 hay .NET 8 luôn là lựa chọn hàng đầu và bắt buộc cho các dự án phần mềm của các doanh nghiệp nhằm đảm bảo hệ thống vận hành an toàn, lâu dài.

Bảng so sánh tổng quan các giai đoạn phát triển của .NET

Tiêu chí so sánh	.NET Framework (2002 - ~2019)	.NET Core (2016 - 2020)	.NET hiện đại (2020 - nay)
Triết lý thiết kế	Mạnh mẽ nhưng khép kín.	Cách mạng, tự do và cởi mở.	Hiện đại, hợp nhất và tối ưu hiệu năng.

Môi trường vận hành	Chỉ phát triển tốt nhất trên Windows.	Chạy ở bất kỳ đâu (Windows, macOS, Linux).	Đa nền tảng đồng nhất dưới một tên gọi duy nhất.
Mô hình kiến trúc	Kiến trúc cũ kiểu "khu vườn có tường bao".	Đập đi xây lại hoàn toàn mới từ con số 0.	Hợp nhất hai thế giới cũ thành một nền tảng duy nhất.

***Điểm cốt lõi cần ghi nhớ:** Hiện tại chúng ta đang được thừa hưởng những gì tinh túy nhất của cả một hành trình tiến hóa. Việc hiểu rõ đặc tính đa nền tảng và ưu tiên chọn các bản LTS chuẩn (như .NET 6, 8) sẽ giúp dự án của bạn vừa đạt hiệu năng tối đa, vừa đảm bảo sự an toàn dài hạn cho cấu trúc phần mềm.*

1.1.2. Các đặc điểm nổi bật của .NET hiện đại

Nếu như lịch sử phát triển cho thấy sự chuyển mình về mặt chiến lược, thì các đặc điểm dưới đây chính là "vũ khí sắc bén" giúp .NET hiện đại trở thành một trong những nền tảng được các tập đoàn công nghệ lớn và lập trình viên săn đón nhất hiện nay.

1. Đa nền tảng (Cross-platform) – "Viết một nơi, chạy mọi nẻo"

- Trước đây, .NET đồng nghĩa với Windows. Nhưng với .NET hiện đại, Microsoft đã loại bỏ hoàn toàn rào cản này. Mã nguồn của bạn sau khi biên dịch sẽ chạy trên một môi trường runtime đồng nhất (được gọi là .NET Runtime) có khả năng tương thích hoàn hảo với mọi hệ điều hành phổ biến: Windows, Linux và macOS. Điều này giúp doanh nghiệp tối ưu hóa chi phí phần cứng và vận hành một cách tối đa.
- **Ví dụ:** Hãy tưởng tượng bạn đang tham gia một dự án xây dựng website thương mại điện tử:
 - **Lúc phát triển (Development):** Bạn dùng máy MacBook (chạy macOS), đồng nghiệp của bạn dùng máy ThinkPad (chạy Windows 11). Cả hai đều có thể mở chung một mã nguồn ASP.NET Core bằng VS Code hoặc Visual Studio để lập trình, test nội bộ mà không bị lệch pha hay xảy ra lỗi xung đột hệ điều hành.
 - **Lúc triển khai (Deployment):** Thay vì phải thuê một server Windows đắt đỏ, doanh nghiệp quyết định đóng gói ứng dụng vào một **Docker Container** chạy hệ điều hành **Linux Ubuntu** để đưa lên đám mây (AWS hoặc Azure). Ứng

dụng vẫn khởi chạy và phục vụ khách hàng một cách mượt mà, trơn tru mà không cần chỉnh sửa lại bất kỳ dòng code nào.

2. Mã nguồn mở (Open-source) – Minh bạch, tự do và hướng về cộng đồng

- Từ chỗ là một công nghệ độc quyền đóng kín, .NET hiện nay đã hoàn toàn "mở cửa" dưới sự quản lý của tổ chức phi lợi nhuận *.NET Foundation*. Toàn bộ mã nguồn cốt lõi của .NET, từ trình biên dịch, thư viện cơ sở đến framework ASP.NET Core, đều được công khai 100% trên GitHub. Điều này mang lại sự minh bạch tuyệt đối, cho phép cộng đồng lập trình viên toàn cầu cùng tham gia phát hiện lỗi bảo mật, tối ưu hóa hiệu năng và đóng góp tính năng mới.
- **Ví dụ:** Khi bạn đang code chức năng xử lý đăng nhập và gặp một lỗi logic rất lạ bên trong thư viện hệ thống của Microsoft (Microsoft.AspNetCore.Identity), thay vì phải chịu bó tay như trước đây với các "hộp đen" mã nguồn đóng:
 - Bạn có thể truy cập thẳng vào kho mã nguồn mở trên GitHub tại địa chỉ github.com/dotnet/aspnetcore.
 - Bạn có thể gõ tìm kiếm đúng đoạn mã đó để đọc xem các kỹ sư của Microsoft đã viết gì, hiểu rõ căn nguyên vận hành của hệ thống.
 - Nếu phát hiện ra một lỗi bảo mật nghiêm trọng hoặc một cách tối ưu thuật toán tốt hơn, bạn hoàn toàn có thể tự tay sửa code và gửi một bản **Pull Request (PR)**. Nếu đóng góp của bạn hữu ích, nó sẽ được duyệt và xuất hiện trong phiên bản .NET chính thức tiếp theo cho hàng triệu lập trình viên thế giới sử dụng.

3. Hiệu năng vượt trội (High-performance) – Tốc độ tối đa, chi phí tối thiểu

- Khi tái cấu trúc từ bản Core, mục tiêu hàng đầu của các kỹ sư Microsoft là đưa ASP.NET Core trở thành một framework siêu nhanh. Nhờ việc tối ưu hóa sâu xuống tầng kiến trúc, quản lý bộ nhớ thông minh (sử dụng các kỹ thuật tiên tiến như `Span<T>`, `Memory<T>` giúp giảm thiểu việc dọn rác - Garbage Collection) và máy chủ web nội bộ **Kestrel** cực kỳ nhẹ, .NET có tốc độ xử lý nhanh đến kinh ngạc.
- **Ví dụ:** Trong các cuộc thử nghiệm độc lập của **TechEmpower Benchmarks** (một tổ chức uy tín chuyên đo lường tốc độ của các framework web trên thế giới), ASP.NET Core liên tục nằm trong nhóm dẫn đầu.

- Nó có khả năng xử lý **hàng triệu request (yêu cầu) mỗi giây** trên một cấu hình phần cứng tiêu chuẩn.
- Khi so sánh với các đối tượng phổ biến khác như Node.js (JavaScript) hay Django (Python), ASP.NET Core cho tốc độ phản hồi nhanh hơn gấp nhiều lần và tiêu tốn ít dung lượng RAM hơn hẳn.
- *Ý nghĩa thực tế:* Đối với một ứng dụng như ví điện tử hay trang tin tức lớn vào giờ cao điểm, việc hệ thống chạy nhanh và tốn ít tài nguyên sẽ giúp doanh nghiệp tiết kiệm hàng ngàn USD tiền thuê tài nguyên điện toán đám mây (Cloud) mỗi tháng.

Bảng tổng kết giá trị cốt lõi của .NET hiện đại

Đặc điểm	Lợi ích cho Lập trình viên	Lợi ích cho Doanh nghiệp
Đa nền tảng	Tự do lựa chọn thiết bị làm việc (Mac, Windows, Linux).	Tiết kiệm chi phí bản quyền, linh hoạt chọn server giá rẻ (Linux).
Mã nguồn mở	Hiểu sâu bản chất code, không bị phụ thuộc vào "hộp đen" công nghệ.	Yên tâm về tính minh bạch, tính bảo mật cao nhờ cộng đồng giám sát.
Hiệu năng cao	Tự tin xây dựng các hệ thống lớn, chịu tải cao mà không lo nghẽn.	Giảm chi phí hạ tầng (RAM/CPU), giữ chân khách hàng nhờ tốc độ phản hồi trang cực nhanh.

Khi bắt tay vào viết những dòng code đầu tiên với ASP.NET Core, bạn hãy luôn nhớ rằng mình đang làm việc với một công nghệ có tiêu chuẩn kỹ thuật hàng đầu thế giới. Việc hiểu rõ các thế mạnh này sẽ giúp bạn biết cách tối ưu hóa cấu trúc thư mục, tận dụng thư viện và tự tin hơn khi thiết kế các hệ thống phần mềm lớn sau này.

1.1.3. Mô hình MVC (Model - View - Controller): Nghệ thuật tổ chức mã nguồn trong ASP.NET Core

Trước hết, bạn cần ghi nhớ một nguyên tắc quan trọng: **MVC không phải là một công nghệ hay ngôn ngữ lập trình**. Nó là một **triết lý kiến trúc (Design Pattern)** nhằm phân chia ứng dụng của bạn thành 3 phần độc lập, mỗi phần đảm nhận một trách

nhiệm riêng biệt. Việc này giúp mã nguồn trở nên gọn gàng, dễ bảo trì và dễ mở rộng.

Để hiểu rõ cách MVC hoạt động, chúng ta hãy cùng phân tích vòng đời của một yêu cầu (Request) từ khi người dùng click chuột cho đến khi trang web hiện ra trên màn hình.

Luồng xử lý chi tiết của một Request trong ASP.NET Core MVC

Hãy tưởng tượng người dùng đang truy cập vào trang web của bạn. Quy trình sẽ diễn ra theo 7 bước tuần tự sau:

1. Request (Yêu cầu khởi tạo)

- Người dùng mở trình duyệt, gõ vào thanh địa chỉ đường dẫn: <https://yoursite.com/products/details/5> và nhấn Enter. Trình duyệt sẽ gửi một HTTP Request đến máy chủ chứa ứng dụng của bạn.

2. Routing (Bộ phân luồng - "Nhân viên tổng đài")

- Middleware Routing của ASP.NET Core là "người" đầu tiên đứng ra chặn cửa nhận request.
- Nó đóng vai trò như một nhân viên tổng đài thông minh, nhìn vào cấu trúc URL và phân tích (bóc tách) để chuyển hướng:
 - products → Sẽ chuyển cuộc gọi này cho bộ phận **ProductsController**.
 - details → Cụ thể là gọi đến hành động (Action) mang tên **Details** bên trong Controller đó.
 - 5 → Kèm theo một dữ liệu đầu vào (tham số/parameter) là 5.

3. Controller (Bộ điều phối - "Người quản lý")

- ProductsController chính thức tiếp nhận cuộc gọi. Lúc này, phương thức Details(int id) được kích hoạt với giá trị id = 5.
- **Trách nhiệm:** Controller giống như một người quản lý phân công công việc. Nó nhận yêu cầu: *"Tôi cần xem chi tiết sản phẩm có ID là 5"*.
- Tuy nhiên, bản thân Controller không tự kết nối database để lấy dữ liệu. Nó biết mình cần phải nhờ đến "chuyên gia" xử lý dữ liệu là **Model**.

4. Model (Lớp Dữ liệu & Nghiệp vụ - "Chuyên gia xử lý")

- Controller gọi đến các lớp nghiệp vụ (ví dụ: ProductRepository hoặc DbContext).

- **Trách nhiệm:** Model đóng vai trò là "chuyên gia nghiệp vụ" dưới hậu trường:
 - Nó tiếp nhận chỉ thị: "Hãy tìm và lấy cho tôi thông tin sản phẩm ID = 5".
 - Nó thực hiện các logic phức tạp: Kết nối tới Cơ sở dữ liệu (SQL Server, MySQL,...), viết câu lệnh truy vấn (hoặc sử dụng Entity Framework Core).
 - Sau khi có dữ liệu, nó đóng gói thông tin (Tên, Giá, Hình ảnh...) thành một đối tượng Product hoàn chỉnh trong C# và trả đối tượng này ngược về cho Controller.

5. Controller (Lần 2: Nhận kết quả và chuyển giao)

- Controller nhận lại đối tượng Product từ Model. Lúc này nhiệm vụ xử lý logic của nó đã gần xong.
- Nó đưa ra quyết định: "Với cục dữ liệu sản phẩm này, tôi sẽ dùng 'khuôn mẫu' có tên là Details.cshtml để hiển thị cho người dùng xem."
- Nó truyền đối tượng Product đó sang cho **View**.

6. View (Giao diện - "Người họa sĩ")

- File Details.cshtml lúc này đóng vai trò như một người họa sĩ lành nghề.
- **Trách nhiệm:** Nó nhận dữ liệu thực tế (đối tượng Product) từ Controller và cầm sẵn một "khung tranh" HTML tĩnh.
- Nó sử dụng cú pháp **Razor** (ví dụ: @Model.Name, @Model.Price) để "vẽ" các dữ liệu động từ C# vào đúng các vị trí tương ứng trên bộ khung HTML.

7. Response (Phản hồi kết quả)

- View hoàn tất việc kết xuất (render). Một trang HTML hoàn chỉnh, mang mã nguồn thuần túy mà trình duyệt có thể hiểu được, đã được tạo ra.
- Máy chủ gửi trả trang HTML này (HTTP Response) ngược về trình duyệt của người dùng. Trình duyệt hiển thị thông tin sản phẩm tuyệt đẹp lên màn hình. Quy trình kết thúc!

Bảng tóm tắt vai trò trong MVC

Thành phần	Vai trò trong ứng dụng	Trách nhiệm chính
------------	------------------------	-------------------

M - Model	Chuyên gia Dữ liệu & Logic	Chứa dữ liệu (Entities) và các quy tắc nghiệp vụ. Tương tác trực tiếp với Database.
V - View	Họa sĩ Giao diện	Hiển thị dữ liệu ra màn hình (HTML, CSS, JS) bằng cách sử dụng cú pháp Razor.
C - Controller	Người quản lý / Điều phối	Lắng nghe Request, gọi Model để lấy dữ liệu, sau đó chọn View phù hợp để trả về.

Điểm cần nhớ: Bằng cách tách biệt này, một bạn lập trình viên chuyên làm giao diện (Front-end) có thể thoải mái sửa file HTML/CSS trong **View** mà không sợ vô tình làm hỏng các đoạn code truy xuất cơ sở dữ liệu rất phức tạp nằm trong **Model**. Đó chính là sức mạnh của nguyên tắc *Separation of Concerns* (Tách biệt các mối quan tâm) mà mô hình MVC mang lại!

1.2. Kiến trúc đa lớp (N-Tier Architecture): Nghệ thuật "Chia để trị"

Khi xây dựng một cái miếu nhỏ, một thợ xây có thể tự làm từ móng đến mái. Nhưng khi xây dựng một tòa nhà chọc trời, bạn cần đội làm móng riêng, thợ điện riêng, thợ nội thất riêng. Phần mềm cũng vậy. Kiến trúc đa lớp sinh ra để giúp chúng ta xây dựng những hệ thống "chọc trời" một cách quy củ, chuyên nghiệp.

1.2.1. Tại sao cần phân tách ứng dụng? Hiểu về Nguyên tắc SoC

Nền tảng của kiến trúc đa lớp (và hầu hết các kỹ thuật phần mềm hiện đại) dựa trên một nguyên tắc: **Separation of Concerns (SoC - Tách biệt các mối quan tâm)**.

- SoC mang một triết lý rất đơn giản nhưng mạnh mẽ: **Mỗi phần của phần mềm chỉ nên làm một việc và làm thật tốt việc đó**. Đừng bắt một đoạn code vừa lo việc hiển thị giao diện đẹp, vừa lo tính toán tiền thuế, lại vừa lo việc kết nối với cơ sở dữ liệu.
- **Ví dụ về việc "Vi phạm" SoC:** Nếu bạn nhồi nhét tất cả (từ mã HTML giao diện, các vòng lặp tính toán giảm giá, đến các câu lệnh SQL INSERT/UPDATE) vào chung một file Controller hoặc một file .cshtml, file đó sẽ trở thành một "nồi lẩu thập cẩm". Khi có lỗi xảy ra, việc tìm kiếm nguyên nhân chẳng khác nào mò kim đáy biển.

Khi áp dụng SoC để chia nhỏ ứng dụng, bạn sẽ nhận được 3 "siêu năng lực":

1. **Dễ bảo trì (Maintainability):** Lỗi ở đâu, sửa ở đó. Giao diện nút bấm bị méo? Bạn tìm ngay ở lớp Presentation (Giao diện). Khách hàng phàn nàn tính sai tiền? Bạn đi thẳng vào lớp Business (Nghệp vụ) để kiểm tra thuật toán. Các phần khác hoàn toàn không bị ảnh hưởng.
2. **Dễ tái sử dụng (Reusability):** Giả sử cốt lõi ứng dụng của bạn (Business Logic) đã được viết và kiểm thử cực kỳ hoàn hảo. Sau này, sếp yêu cầu làm thêm app Mobile (iOS/Android) hoặc Desktop. Bạn không cần phải viết lại các công thức tính toán nữa, ứng dụng Mobile chỉ việc tái sử dụng lại nguyên vẹn lớp Business Logic hiện có.
3. **Dễ làm việc nhóm (Teamwork & Parallel Development):** Trong một team lớn, team Frontend (chuyên HTML/CSS/JS) có thể thiết kế giao diện độc lập. Cùng lúc đó, team Backend (chuyên C#/SQL) có thể tập trung tối ưu hóa các lớp Nghiệp vụ và Truy xuất dữ liệu mà không sợ dẫm chân lên nhau.

1.2.2. Phân tích chi tiết kiến trúc 3 lớp (3-Tier) qua ví dụ "Nhà Hàng"

Để dễ hình dung nhất, hãy tưởng tượng một dự án phần mềm giống hệt như cách vận hành của một nhà hàng 5 sao. Sự chuyên môn hóa được chia thành 3 lớp (3 Layers/Tiers) rõ rệt:

Lớp 1: Presentation Layer (Lớp Giao diện / Lớp Trình bày)

- **Mối quan tâm chính:** Trực tiếp giao tiếp với người dùng. Chịu trách nhiệm hiển thị dữ liệu đẹp mắt và tiếp nhận các thao tác (click, gõ chữ) từ khách hàng.
- **Ví dụ nhà hàng:** Đây là **Khu vực phục vụ khách (Dining Room)**. Nhân viên phục vụ mang menu ra, cười tươi ghi nhận món ăn khách order, sau đó bưng đĩa thức ăn được trang trí đẹp mắt lên bàn. Người phục vụ tuyệt đối không xông vào bếp để xào nấu.
- **Trong Project SportsStore:** Nó tương ứng với project SportsStore.WebUI. Nơi đây chỉ chứa Controllers (nhận yêu cầu) và Views (file HTML/CSS/Razor để hiển thị cho người xem).

Lớp 2: Business Logic Layer (BLL - Lớp Nghiệp vụ)

- **Mối quan tâm chính:** Đây là bộ não và trái tim của hệ thống. Nó chứa toàn bộ các quy tắc nghiệp vụ, các thuật toán tính toán (ví dụ: mua 3 sản phẩm thì giảm

giá 10%) và kiểm tra tính hợp lệ của dữ liệu (xác thực người dùng có đủ tiền để mua không).

- **Ví dụ nhà hàng:** Đây là **Bếp trưởng**. Bếp trưởng tiếp nhận order từ phục vụ, biết chính xác công thức bí mật để nấu món bò bít tết. Bếp trưởng quyết định món ăn phải có đủ các gia vị nào thì mới đạt tiêu chuẩn nhà hàng. Nhưng bếp trưởng không tự đi ra siêu thị hay kho để khuôn vác thịt bò.
- **Trong Project SportsStore:** Nó tương ứng với project SportsStore.Domain. Nơi đây chứa các Đối tượng cốt lõi (Entities như Product.cs, Order.cs) và các "hợp đồng" quy định luật chơi (Interfaces như IProductRepository).

Lớp 3: Data Access Layer (DAL - Lớp Truy cập Dữ liệu)

- **Mối quan tâm chính:** Giao tiếp trực tiếp với nguồn lưu trữ vật lý (như SQL Server, MySQL, hoặc File hệ thống). Nó chỉ lo việc đọc dữ liệu lên hoặc ghi dữ liệu xuống một cách an toàn và tối ưu nhất.
- **Ví dụ nhà hàng:** Đây là **Thủ kho / Kho chứa nguyên liệu**. Thủ kho nhận danh sách từ Bếp trưởng, đi vào kho lạnh lớn, biết chính xác kệ số mấy chứa thịt bò, tủ nào chứa rau. Lấy đúng 5kg thịt bò giao cho bếp trưởng. Thủ kho không cần biết bếp trưởng nấu món gì, và cũng không bao giờ gặp mặt khách hàng.
- **Trong Project SportsStore:** Nó tương ứng với project SportsStore.Infrastructure. Nơi đây chứa các lớp chịu trách nhiệm truy cập Database (như EFProductRepository sử dụng công cụ Entity Framework Core để chạy lệnh SQL).

Bảng tóm tắt Kiến trúc 3 Lớp

Tên lớp (Layer/Tier)	Tên Project (SportsStore)	Vai trò (Phần mềm)	Vai trò (Nhà hàng)	Quy tắc ngầm
1. Presentation	SportsStore.WebUI	Giao diện, Controller, Routing	Phục vụ bàn	Chỉ gọi đến lớp BLL. Không bao giờ gọi thẳng DAL.

2. Business Logic	SportsStore.Domain	Entities, Interfaces, Xử lý nghiệp vụ	Bếp trưởng	Trái tim độc lập. Không phụ thuộc vào lớp nào khác.
3. Data Access	SportsStore.Infrastructure	Entity Framework, SQL, Database	Thủ kho	Chỉ thực thi các Interface do lớp BLL định ra.

Lời khuyên cho lập trình viên: Việc hiểu rõ nguyên tắc phân lớp này sẽ cứu bạn khỏi những dự án "ác mộng" trong tương lai. Khi bạn thiết kế được các ranh giới rõ ràng, phần mềm của bạn có thể dễ dàng bảo trì trong vòng 5-10 năm tới mà không sợ bị "vỡ trận" khi thêm tính năng mới!

1.3. Cấu trúc Solution & Project trong .NET

Khi bước vào xây dựng một dự án phần mềm thực tế bằng .NET, việc tổ chức mã nguồn một cách khoa học ngay từ đầu là vô cùng quan trọng. Để quản lý hàng trăm file code mà không bị rối, .NET sử dụng hai khái niệm cốt lõi: **Solution** và **Project**.

1.2.3. Solution vs. Project: Ẩn dụ về "Cái Tủ" và "Ngăn Tủ"

Để những khái niệm kỹ thuật này trở nên gần gũi, chúng ta hãy hình dung toàn bộ mã nguồn của ứng dụng "Cửa hàng thể thao" (SportsStore) giống như một chiếc tủ hồ sơ lớn trong văn phòng.

- Solution (File đuôi .sln): Chiếc tủ hồ sơ tổng
 - **Bản chất:** Solution đóng vai trò là chiếc tủ lớn nhất, là container chứa và quản lý tất cả các thành phần liên quan đến dự án "Cửa hàng thể thao". Bản thân file .sln không chứa mã nguồn cụ thể, nó chỉ chứa các chỉ mục và đường dẫn để liên kết các ngăn tủ lại với nhau.
- Project (File đuôi .csproj): Các ngăn tủ chuyên dụng
 - **Bản chất:** Mỗi Project là một ngăn tủ độc lập nằm bên trong Solution. Mỗi ngăn tủ này chứa một nhóm tài liệu (các file code .cs, cấu hình) có cùng chức

năng, nhiệm vụ riêng biệt và khi biên dịch sẽ cho ra một sản phẩm chạy được (.exe) hoặc một thư viện (.dll).

Phân tích các "Ngăn tủ Project" trong dự án SportsStore:

Tên Ngăn Tủ (Project)	Định dạng file	Vai trò trong hệ thống	Ví dụ file cụ thể bên trong
SportsStore.Domain	.csproj	Ngăn chứa Bản thiết kế & Quy tắc: Chứa các thực thể cốt lõi và định nghĩa các hành vi mà hệ thống phải có.	Product.cs (Định nghĩa sản phẩm gồm tên, giá...), IProductRepository.cs (Hợp đồng quy định cách lấy dữ liệu).
SportsStore.Infrastructure	.csproj	Ngăn chứa Công cụ vật lý: Chứa các công cụ kỹ thuật để làm việc với phần cứng, cơ sở dữ liệu hoặc các dịch vụ bên ngoài.	DataContext.cs (Cấu hình kết nối SQL Server), EFProductRepository.cs (Code hiện thực việc lấy sản phẩm từ DB).
SportsStore.WebUI	.csproj	Ngăn chứa Phòng trưng bày: Nơi chứa giao diện người dùng và các nhân viên tiếp đón, điều phối luồng dữ liệu.	HomeController.cs (Tiếp nhận yêu cầu từ khách), Index.cshtml (Giao diện hiển thị danh sách sản phẩm).

1.2.4. Project References: "Luồng Quyền Lực" trong Kiến trúc Phần mềm

Trong một tổ chức, các phòng ban cần phải tương tác với nhau để hoàn thành công việc. Trong .NET, việc này được thực hiện thông qua **Project References (Tham chiếu**

giữa các dự án). Khi một Project A tham chiếu đến Project B, điều đó có nghĩa là Project A có quyền sử dụng tất cả các class công khai nằm bên trong Project B.

Luật Vàng của Kiến trúc 3 Lớp: > Sự phụ thuộc chỉ được phép đi theo **MỘT CHIỀU DUY NHẤT** từ ngoài vào trong:

Presentation Layer (WebUI) ➡ Business Logic (Domain) ⬅ Data Access (Infrastructure).

Phân tích chi tiết luồng mũi tên phụ thuộc:

1. Mũi tên từ SportsStore.WebUI ➡ SportsStore.Domain

- **Giải thích:** "Phòng trưng bày" (WebUI) muốn bán được hàng thì nhân viên bán hàng (Controller) phải biết rõ hình dáng, kích thước và thông tin của sản phẩm (Product.cs nằm trong Domain). Do đó, WebUI bắt buộc phải tham chiếu đến Domain để sử dụng các định nghĩa thực thể này.

2. Mũi tên từ SportsStore.WebUI ➡ SportsStore.Infrastructure

- **Giải thích:** Khi khách hàng nhấn nút "Mua hàng" trên giao diện, nhân viên điều phối (Controller ở lớp WebUI) cần một người xuống kho lấy sản phẩm lên. Người xuống kho chính là lớp Infrastructure. Vì vậy, WebUI phải tham chiếu tới Infrastructure để kích hoạt các công cụ truy xuất cơ sở dữ liệu.

3. Mũi tên từ SportsStore.Infrastructure ➡ SportsStore.Domain

- **Giải thích:** "Người thủ kho" (Infrastructure) đảm nhận việc bốc dỡ dữ liệu từ SQL Server lên. Để bốc đúng hàng và đóng gói đúng chuẩn, người thủ kho phải nhìn vào bản thiết kế của sản phẩm nằm trong Domain. Do đó, Infrastructure phải phụ thuộc vào Domain.

Điều quan trọng nhất: TUYỆT ĐỐI KHÔNG CÓ MŨI TÊN ĐI RA TỪ DOMAIN

Ngăn tử **Domain (Business Logic)** là cốt lõi, là tài sản trí tuệ quan trọng nhất của doanh nghiệp. Nó phải hoàn toàn độc lập, tinh khiết và **không được phép tham chiếu đến bất kỳ project nào khác**.

Hệ quả của việc vi phạm luật vàng (Circular Dependency - Tham chiếu vòng quanh):

Hãy tưởng tượng nếu bạn phá vỡ quy tắc, cho phép SportsStore.Domain tham chiếu ngược lại SportsStore.WebUI. Việc này giống như việc bạn bắt *Bản thiết kế cốt lõi của sản phẩm* phải phụ thuộc vào *Màu sơn của phòng trưng bày*.

Hôm sau, nếu đội Frontend quyết định đổi màu sơn phòng trưng bày từ Xanh sang

Đỏ (sửa code ở lớp WebUI), hệ thống sẽ bắt bạn phải đem Bản thiết kế sản phẩm (Domain) ra để sửa đổi và biên dịch lại. Điều này cực kỳ vô lý! Nó làm cho hệ thống trở nên cứng nhắc, các lớp dính chặt vào nhau (High Coupling), mất đi hoàn toàn khả năng bảo trì và tái sử dụng code.

HƯỚNG DẪN DEMO

Cách 1: Sử dụng Visual Studio 2022 (Giao diện trực quan)

Bước 1: Khởi tạo Solution trống (Tạo "Cái tủ hồ sơ")

Mục đích của bước này là tạo ra một tệp .sln và một thư mục gốc rỗng để chứa tất cả các project sau này.

1. **Mở Visual Studio 2022**, tại màn hình khởi động, chọn **Create a new project** (Tạo dự án mới).
2. Trên thanh tìm kiếm ở góc trên cùng, hãy gõ từ khóa Blank Solution.
3. Chọn template **Blank Solution** có biểu tượng hình trang giấy trắng và nhấn **Next**.
4. Tại màn hình cấu hình:
 - **Solution name:** Gõ tên dự án của bạn (Ví dụ: SportsStore).
 - **Location:** Chọn thư mục lưu trữ trên máy tính của bạn (Ví dụ: D:\Projects\).
5. Nhấn **Create**.

Lúc này, bạn nhìn sang cửa sổ **Solution Explorer** (thường ở góc phải màn hình), bạn sẽ thấy một Solution rỗng mang tên SportsStore đã sẵn sàng.

Bước 2: Tạo các Project Thư viện (Class Library) cho Domain và Infrastructure

Class Library (Thư viện lớp) là loại project khi chạy (build) sẽ tạo ra file .dll. Nó không có giao diện, không tự chạy được (không có nút Play), mà chỉ chứa code nghiệp vụ để các project khác gọi đến.

A. Tạo project SportsStore.Domain:

1. Click chuột phải vào tên Solution SportsStore trong Solution Explorer.
2. Chọn Add ➡ New Project...
3. Gõ từ khóa Class Library vào ô tìm kiếm. Chọn template **Class Library** (đảm bảo nó có chữ C# và hỗ trợ Linux/macOS/Windows) ➡ Nhấn **Next**.

4. **Project name:** Đặt tên là SportsStore.Domain ➡ Nhấn **Next**.
5. **Framework:** Chọn phiên bản .NET 8.0 (Long Term Support) ➡ Nhấn **Create**.
6. (Tùy chọn) Visual Studio sẽ tự tạo một file Class1.cs. Bạn có thể xóa file này đi để sau này tự tạo các file thực thể (như Product.cs) của riêng mình.

B. Tạo project SportsStore.Infrastructure:

- Bạn lặp lại y hệt 6 thao tác trên, nhưng ở bước đặt tên (Project name), hãy đổi thành SportsStore.Infrastructure.

Bước 3: Tạo Project WebUI (Ứng dụng Web MVC)

Đây sẽ là project có thể chạy trực tiếp trên trình duyệt, chứa giao diện và đóng vai trò tiếp nhận yêu cầu từ người dùng.

1. Tiếp tục click chuột phải vào Solution SportsStore ➡ Chọn **Add** ➡ **New Project...**
2. Lần này, gõ vào ô tìm kiếm: ASP.NET Core Web App (Model-View-Controller).
3. Chọn đúng template này (biểu tượng chữ C# và quả cầu) ➡ Nhấn **Next**.
4. **Project name:** Đặt tên là SportsStore.WebUI ➡ Nhấn **Next**.
5. **Framework:** Vẫn chọn **.NET 8.0 (Long Term Support)**. Các tùy chọn khác giữ nguyên mặc định ➡ Nhấn **Create**.

Ngay lập tức, Visual Studio sẽ sinh ra project WebUI với đầy đủ cấu trúc thư mục chuẩn của MVC bao gồm: Controllers, Models, Views, wwwroot (chứa CSS/JS), và file cấu hình Program.cs.

Bước 4: Thiết lập Tham chiếu (Project References) - Khai báo "Luồng Quyền Lực"

Nếu dừng ở bước 3, ba project này vẫn là 3 ngăn tủ độc lập, không ai biết ai. Chúng ta cần thiết lập tham chiếu để báo cho trình biên dịch (compiler) biết project nào được phép mượn code của project nào. Thao tác này sẽ ghi đè cấu hình vào file .csproj.

A. Cho WebUI tham chiếu đến Domain và Infrastructure:

1. Mở rộng project SportsStore.WebUI trong Solution Explorer.
2. Click chuột phải vào mục **Dependencies** (hoặc References) ➡ Chọn **Add Project Reference...**
3. Một cửa sổ hiện ra liệt kê các project có trong Solution. Bạn hãy **tick (chọn)** vào

2 ô: SportsStore.Domain và SportsStore.Infrastructure.

4. Nhấn **OK**. (Lúc này, phòng trưng bày WebUI đã có thể gọi nhân viên từ kho Infrastructure và đọc bản thiết kế Domain).

B. Cho Infrastructure tham chiếu đến Domain:

1. Mở rộng project SportsStore.Infrastructure.
2. Click chuột phải vào mục Dependencies ➡ Chọn Add Project Reference...
3. **Tick (chọn)** vào ô: SportsStore.Domain.
4. Nhấn **OK**. (Lúc này, thủ kho Infrastructure đã có thể đọc bản thiết kế hàng hóa từ Domain).

***Nhắc lại nguyên tắc sống còn:** Tuyệt đối KHÔNG click chuột phải vào Dependencies của SportsStore.Domain để tham chiếu đến ai cả. Lớp Domain phải hoàn toàn trống trơn trong mục Project References!*

C. Đặt WebUI làm Project khởi chạy mặc định:

- Để khi nhấn nút "Play" (Start) ứng dụng sẽ chạy ngay trang web, bạn click chuột phải vào project SportsStore.WebUI ➡ Chọn **Set as Startup Project**. Tên project này sẽ được in đậm lên.

Cách 2: Sử dụng VS Code + .NET CLI (Dòng lệnh)

Việc sử dụng giao diện dòng lệnh (CLI - Command Line Interface) thay vì click chuột có vẻ khô khan lúc ban đầu, nhưng lại mang đến lợi ích vô cùng to lớn. Nó giúp bạn hiểu sâu sắc bản chất hệ thống thực sự đang làm gì dưới nền, đồng thời đây là kỹ năng **bắt buộc phải có** khi bạn muốn triển khai dự án tự động (CI/CD) hoặc làm việc trên môi trường máy chủ Linux.

Dưới đây là các bước thao tác trên Terminal (hoặc Command Prompt/PowerShell) để xây dựng lại chính xác kiến trúc 3 lớp mà chúng ta vừa làm trên Visual Studio:

Bước 1: Chuẩn bị không gian làm việc

1. Mở Terminal (hoặc Command Prompt).
2. Tạo một thư mục gốc cho dự án (ví dụ: thư mục SportsStore) và dùng lệnh cd SportsStore để di chuyển vào thư mục đó.

Bước 2: Tạo file Solution ("Chiếc tủ hồ sơ")

Gõ lệnh sau vào Terminal và nhấn Enter:

```
dotnet new sln -n SportsStore
```

- dotnet: Tiền tố bắt buộc để gọi công cụ .NET CLI.
- new sln: Yêu cầu tạo một file project mới thuộc loại "Solution" (khung quản lý dự án).
- -n SportsStore: Cờ -n (viết tắt của --name) dùng để chỉ định tên cho file. Kết quả là file SportsStore.sln sẽ được sinh ra.

Bước 3: Tạo các Project ("Các ngăn tủ chuyên dụng")

Chúng ta sẽ tạo lần lượt 3 project cho 3 lớp. Lệnh này vừa tạo project, vừa tạo luôn thư mục con để chứa code cho gọn gàng.

1. Tạo lớp Nghiệp vụ (Domain) và Lớp Truy cập Dữ liệu (Infrastructure):

```
dotnet new classlib -n SportsStore.Domain -o SportsStore.Domain  
dotnet new classlib -n SportsStore.Infrastructure -o  
SportsStore.Infrastructure
```

- new classlib: Tạo một project thuộc loại *Class Library* (Thư viện lớp, không có giao diện, khi build ra file .dll). Hoàn hảo cho Domain và Infrastructure.
- -o SportsStore...: Cờ -o (viết tắt của --output) yêu cầu .NET tạo ra một thư mục con cùng tên và đặt toàn bộ mã nguồn của project vào trong đó, tránh việc các file bị trộn lẫn vào nhau ở thư mục gốc.

2. Tạo lớp Giao diện (WebUI):

```
dotnet new mvc -n SportsStore.WebUI -o SportsStore.WebUI
```

- new mvc: Tạo một project ứng dụng web sử dụng khuôn mẫu *ASP.NET Core Web App (Model-View-Controller)*. Lệnh này sẽ tự động sinh ra cấu trúc thư mục đặc thù: Controllers, Models, Views,...

Bước 4: Thêm các Project vào Solution ("Cất ngăn vào tủ")

Lúc này, các project đã được tạo ra trên ổ cứng nhưng file Solution SportsStore.sln chưa hề biết đến sự tồn tại của chúng. Bạn cần liên kết chúng lại bằng lệnh:

```
dotnet sln add SportsStore.Domain  
dotnet sln add SportsStore.Infrastructure
```

```
dotnet sln add SportsStore.WebUI
```

- `sln add <đường_dẫn>`: Lệnh này mở file `.sln` ra và ghi thêm đường dẫn của các project (`.csproj`) vào trong đó. Giờ đây, khi bạn mở Solution bằng VS Code hoặc Visual Studio, toàn bộ 3 project sẽ hiện lên cùng một nơi.

Bước 5: Thiết lập Tham chiếu - Xây dựng "Luồng Quyền Lực"

Đây là bước quan trọng nhất để tuân thủ nguyên tắc Kiến trúc 3 lớp. Chúng ta cần định nghĩa sự phụ thuộc giữa các project.

1. Cấp quyền cho WebUI sử dụng Domain và Infrastructure:

```
dotnet add SportsStore.WebUI reference SportsStore.Domain
dotnet add SportsStore.WebUI reference SportsStore.Infrastructure
```

2. Cấp quyền cho Infrastructure sử dụng Domain:

```
dotnet add SportsStore.Infrastructure reference SportsStore.Domain
```

- `dotnet add <PROJECT_A> reference <PROJECT_B>`: Lệnh này thao tác trực tiếp trên file cấu hình `.csproj` của Project A.
- Nó chèn thêm một thẻ `<ProjectReference>` trỏ tới Project B. Về mặt ý nghĩa, lệnh này báo với hệ thống rằng: *"Khi biên dịch Project A, hãy đảm bảo rằng Project B cũng được biên dịch và cho phép Project A gọi đến các đoạn code của Project B"*.

Sau khi gõ xong toàn bộ các lệnh trên, bạn chỉ cần gõ code . trong Terminal. Trình soạn thảo Visual Studio Code sẽ tự động mở lên với một cấu trúc thư mục 3 lớp chuyên nghiệp, hoàn hảo và đã được liên kết chặt chẽ với nhau, sẵn sàng để bạn code những tính năng đầu tiên!

CÂU HỎI LÝ THUYẾT Củng Cố

1. Đặc điểm quan trọng nhất của .NET Core (và .NET 5+) so với .NET Framework là gì?
2. Trong mô hình MVC, thành phần nào chịu trách nhiệm chính trong việc xử lý logic nghiệp vụ và tương tác với dữ liệu?
3. Hãy giải thích nguyên tắc "Tách biệt các mối quan tâm" (SoC) bằng một ví dụ của riêng bạn.
4. Trong kiến trúc 3 lớp của dự án SportsStore, tại sao project SportsStore.Domain không được phép tham chiếu đến SportsStore.WebUI? Điều gì sẽ xảy ra nếu

chúng ta phá vỡ quy tắc này?

TỔNG KẾT BÀI HỌC

Tuần này, chúng ta đã xây dựng một nền móng vững chắc. Bạn đã hiểu được "tại sao" đằng sau các quyết định kiến trúc:

- Chúng ta dùng **.NET 8** vì nó hiện đại, đa nền tảng và hiệu năng cao.
- Chúng ta dùng **MVC** để tổ chức code trong ứng dụng web một cách khoa học.
- Chúng ta dùng **kiến trúc 3 lớp** để đảm bảo ứng dụng dễ bảo trì, dễ mở rộng và linh hoạt trước các thay đổi trong tương lai.
- Chúng ta đã biết cách hiện thực hóa kiến trúc này bằng cách tạo **Solution** và các **Project** liên quan, đồng thời thiết lập đúng **luồng phụ thuộc** giữa chúng.

BÀI TẬP VỀ NHÀ

1. Thực hiện lại toàn bộ các bước trong phần "Hướng dẫn demo" để tự tay tạo cấu trúc dự án SportsStore từ đầu. Khuyến khích thử cả 2 cách (Visual Studio và .NET CLI) để làm quen.
2. Vẽ lại sơ đồ kiến trúc 3 lớp và sơ đồ tham chiếu project của dự án SportsStore ra giấy.
3. (Nâng cao) Tìm hiểu và trả lời ngắn gọn: Sự khác biệt giữa kiến trúc 3 lớp (3-Tier) và kiến trúc 3 tầng (3-Layer) là gì?

TÀI LIỆU THAM KHẢO

- [1]. Tổng quan về ASP.NET Core: [Microsoft Docs - Introduction to ASP.NET Core](#)
- [2]. Tổng quan về Giải pháp và Dự án: [Microsoft Docs - Solutions and Projects in Visual Studio](#)
- [3]. Lệnh dotnet: [Microsoft Docs - dotnet command](#)